Bharatiya Vidya Bhavan's

SARDAR PATEL INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering — Cryptography and Network Security Laboratory

Experiment: Secure E-mail using PGP and S/MIME

Field	Value
Name	Anusha Dalal
UID	2024300032
Date of experiment	31 Oct 2026

Aim

To implement secure message exchange using Pretty Good Privacy (PGP/OpenPGP) and S/MIME, demonstrating public-key management, digital signatures, confidentiality through encryption, signature verification, decryption, and detection of message tampering.

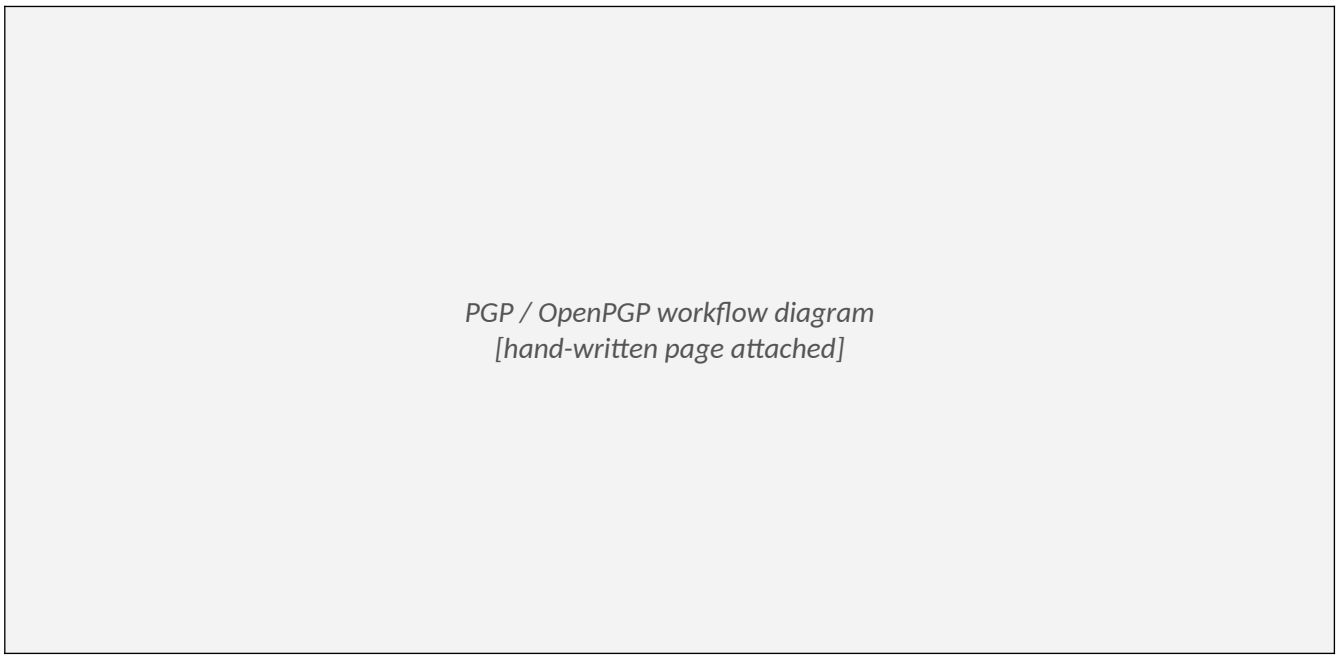
Contents (order of Section 8)

#	Item
1	Hand-drawn PGP and S/MIME workflow diagrams
2	Table 1: execution environment
3	PGP evidence: key listings and fingerprints, export/import, detached signature, encryption/decryption, signed-and-encrypted message, tampering
4	S/MIME evidence: CA and user certificates, signed message, verification, encrypted message, decryption, tampering and untrusted-certificate results
5	Tables 2 and 3, comparative analysis, answers to the discussion questions, conclusion
6	Commands used and deviations caused by software versions

Every terminal capture shows the name and UID line and the prompt of the machine used (root@vm). No private keys are included in this report.

1. Hand-drawn workflow diagrams (Part A)

Drawn on paper before using the computer: Sender (Alice), Receiver (Bob), the insecure channel, key pairs/certificates, signing, encryption, decryption, verification, the ACCEPT/REJECT decision, an attacker modifying the message, and private keys never transmitted.

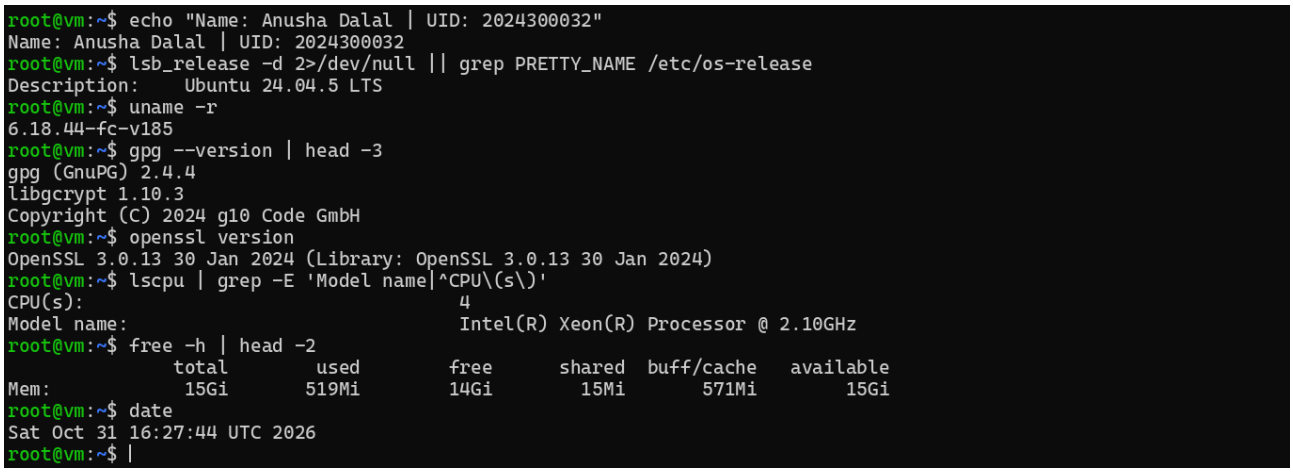




2. Table 1: Execution environment

Parameter	Observed value
Operating system and version	Ubuntu 24.04.5 LTS (kernel 6.18.44-fc-v185)
GnuPG version	GnuPG 2.4.4 (libgcrypt 1.10.3)
OpenSSL version	OpenSSL 3.0.13 30 Jan 2024
CPU / RAM (optional)	Intel(R) Xeon(R) Processor @ 2.10GHz, 4 logical CPUs / 15 GiB
Date of experiment	Sat Oct 31 16:27:44 UTC 2026

Figure 1: Table 1 evidence: operating system, kernel, GnuPG and OpenSSL versions, CPU, RAM, date



3. PGP / OpenPGP evidence (Part B)

Alice and Bob each have a separate GnuPG home directory, so each private key stays in its own keyring. Only public keys, the message, the signature and encrypted files are placed in the transfer folder.

Figure 2: Step B1: GnuPG installed - gpg --version

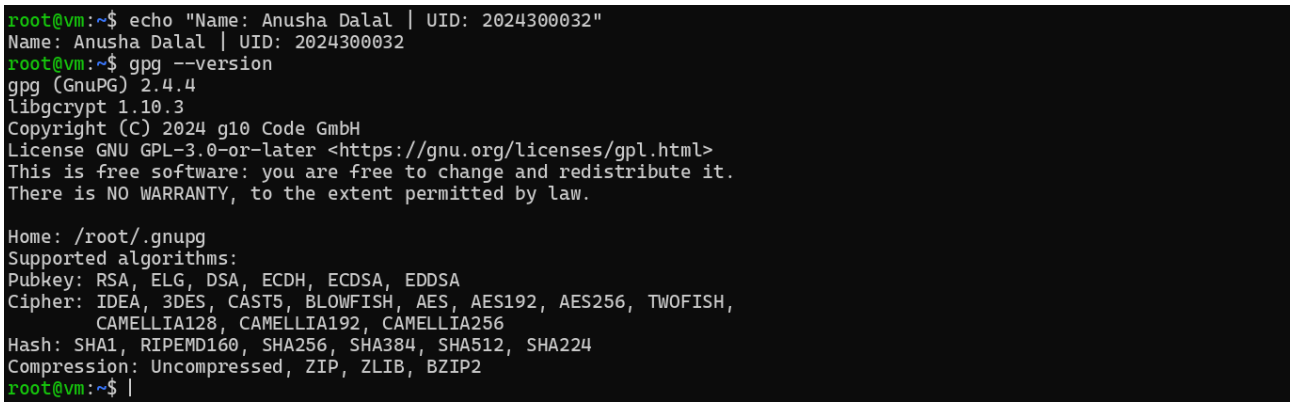


Figure 3: Step B2: separate working directories and GnuPG homes for Alice and Bob

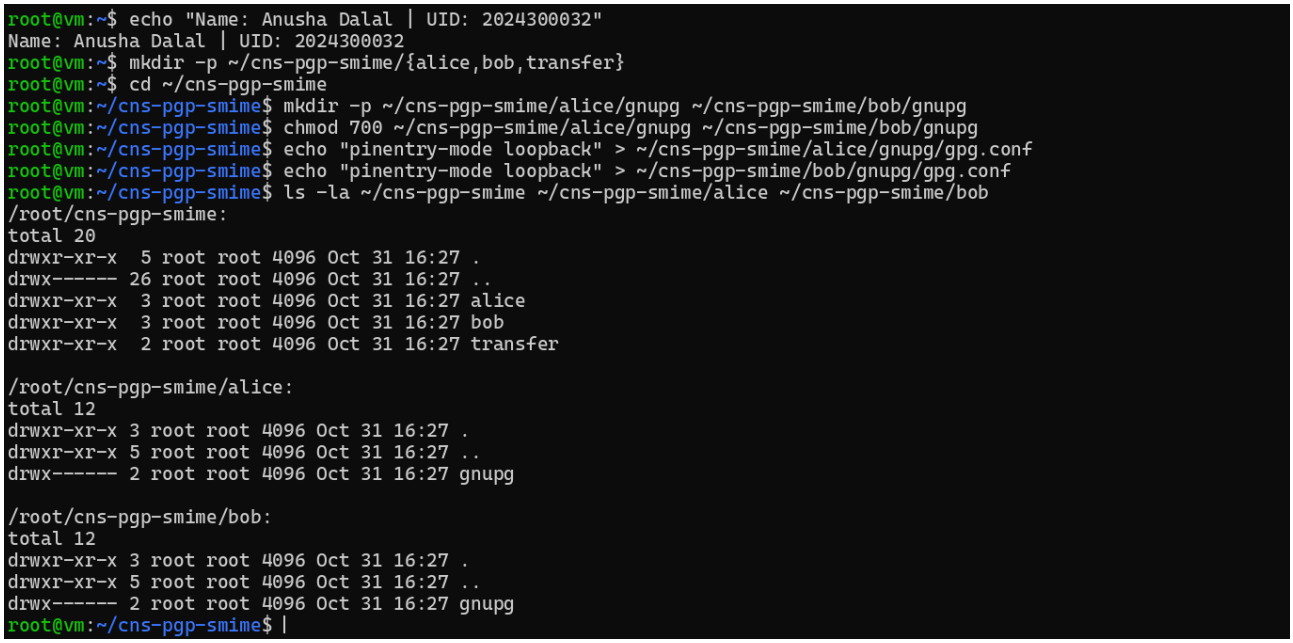


Figure 4: Step B3: Alice's key pair (RSA and RSA, 3072 bits) - gpg --full-generate-key, list-keys, list-secret-keys (1/2)

```
root@vm:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
root@vm:~/cns-pgp-smime$ gpg --full-generate-key
gpg (GnuPG) 2.4.4; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: keybox '/root/cns-pgp-smime/alice/gnupg/pubring.kbx' created
Please select what kind of key you want:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 3072
Requested keysize is 3072 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Sun Oct 31 16:27:44 2027 UTC
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Alice CNS Lab
Email address: alice-lab@example.test
Comment:
You selected this USER-ID:
    "Alice CNS Lab <alice-lab@example.test>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? O
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
Enter passphrase:                   We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.

gpg: /root/cns-pgp-smime/alice/gnupg/trustdb.gpg: trustdb created
gpg: directory '/root/cns-pgp-smime/alice/gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/root/cns-pgp-smime/alice/gnupg/openpgp-revocs.d/B350D8CB3960E304F4ECD9CB01BC
8D44B88947A0.rev'
public and secret key created and signed.

pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      B350D8CB3960E304F4ECD9CB01BC8D44B88947A0
uid                      Alice CNS Lab <alice-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
root@vm:~/cns-pgp-smime$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
```

Figure 5: Step B3: Alice's key pair (RSA and RSA, 3072 bits) - gpg --full-generate-key, list-keys, list-secret-keys (2/2)

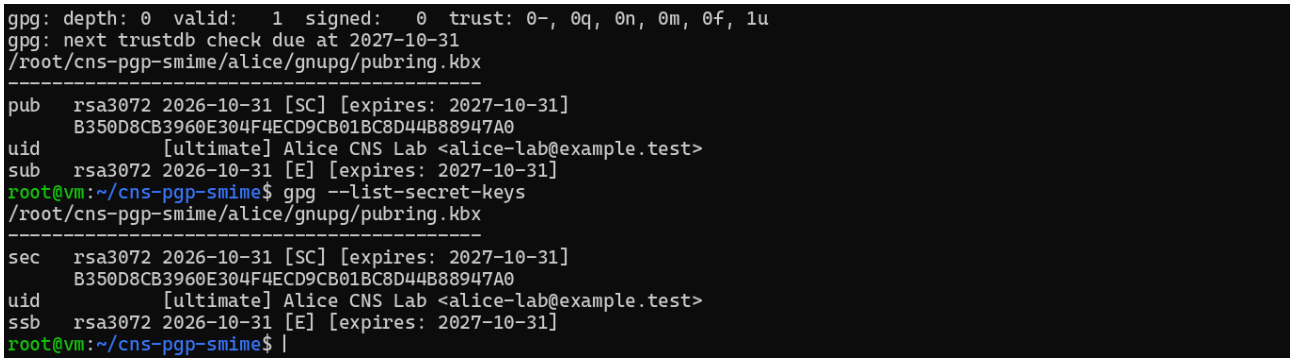


Figure 6: Step B3: Bob's key pair (RSA and RSA, 3072 bits) and key listing (1/2)

```
root@vm:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
root@vm:~/cns-pgp-smime$ gpg --full-generate-key
gpg (GnuPG) 2.4.4; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: keybox '/root/cns-pgp-smime/bob/gnupg/pubring.kbx' created
Please select what kind of key you want:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 3072
Requested keysize is 3072 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Sun Oct 31 16:27:48 2027 UTC
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Bob CNS Lab
Email address: bob-lab@example.test
Comment:
You selected this USER-ID:
    "Bob CNS Lab <bob-lab@example.test>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? O
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
Enter passphrase:
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /root/cns-pgp-smime/bob/gnupg/trustdb.gpg: trustdb created
gpg: directory '/root/cns-pgp-smime/bob/gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/root/cns-pgp-smime/bob/gnupg/openpgp-revocs.d/7A850240F9ECA7B028E82D4CC19056
78CBE1BC88.rev'
public and secret key created and signed.

pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      7A850240F9ECA7B028E82D4CC1905678CBE1BC88
uid                      Bob CNS Lab <bob-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
root@vm:~/cns-pgp-smime$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
```

Figure 7: Step B3: Bob's key pair (RSA and RSA, 3072 bits) and key listing (2/2)

```
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: next trustdb check due at 2027-10-31
/root/cns-pgp-smime/bob/gnupg/pubring.kbx
------------------------------------------
pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      7A850240F9ECA7B028E82D4CC1905678CBE1BC88
uid           [ultimate] Bob CNS Lab <bob-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
root@vm:~/cns-pgp-smime$
```

Figure 8: Step B4: export public keys only, import into the other keyring, compare fingerprints

```
root@vm:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
root@vm:~/cns-pgp-smime$ gpg --armor --export alice-lab@example.test > ~/cns-pgp-smime/alice_public.asc
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
root@vm:~/cns-pgp-smime$ gpg --armor --export bob-lab@example.test > ~/cns-pgp-smime/bob_public.asc
root@vm:~/cns-pgp-smime$ ls -l ~/cns-pgp-smime/*.asc
-rw-r--r-- 1 root root 2464 Oct 31 16:27 /root/cns-pgp-smime/alice_public.asc
-rw-r--r-- 1 root root 2460 Oct 31 16:27 /root/cns-pgp-smime/bob_public.asc
root@vm:~/cns-pgp-smime$ head -n 4 ~/cns-pgp-smime/alice_public.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQGNBGrmFwEBDADty5voOTmqY3AE6DiV3PtrCOqYmz5udpLZ6Ke+qFP45w2Ptgcg
y2kdtW+OBmcsEC2Y3kTwnKRizOACh9cta4zJ49djjwn7gy/5NAUHq6VAhyqt5E0U
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
root@vm:~/cns-pgp-smime$ gpg --import ~/cns-pgp-smime/bob_public.asc
gpg: key C1905678CBE1BC88: public key "Bob CNS Lab <bob-lab@example.test>" imported
gpg: Total number processed: 1
gpg:               imported: 1
root@vm:~/cns-pgp-smime$ gpg --fingerprint bob-lab@example.test
pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      7A85 0240 F9EC A7B0 28E8  2D4C C190 5678 CBE1 BC88
uid           [ unknown] Bob CNS Lab <bob-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
root@vm:~/cns-pgp-smime$ gpg --import ~/cns-pgp-smime/alice_public.asc
gpg: key 01BC8D44B88947A0: public key "Alice CNS Lab <alice-lab@example.test>" imported
gpg: Total number processed: 1
gpg:               imported: 1
root@vm:~/cns-pgp-smime$ gpg --fingerprint alice-lab@example.test
pub   rsa3072 2026-10-31 [SC] [expires: 2027-10-31]
      B350 D8CB 3960 E304 F4EC  D9CB 01BC 8D44 B889 47A0
uid           [ unknown] Alice CNS Lab <alice-lab@example.test>
sub   rsa3072 2026-10-31 [E] [expires: 2027-10-31]
root@vm:~/cns-pgp-smime$
```

Figure 9: Step B5: message created and signed by Alice (detached ASCII-armored signature)

```
root@vm:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime$ printf 'CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01.\n' > ~/cns-pgp-smime/messa
ge.txt
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
root@vm:~/cns-pgp-smime$ gpg --armor --local-user alice-lab@example.test --detach-sign --output ~/cns-pgp-smime/mess
age.txt.asc ~/cns-pgp-smime/message.txt
Enter passphrase:
root@vm:~/cns-pgp-smime$ cp ~/cns-pgp-smime/message.txt* ~/cns-pgp-smime/transfer/
root@vm:~/cns-pgp-smime$ cat ~/cns-pgp-smime/transfer/message.txt ~/cns-pgp-smime/transfer/message.txt.asc
CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01.
-----BEGIN PGP SIGNATURE-----

iQHLBAABCgA1FiEEs1DYyzlg4wT07NnLAbyNRLiJR6AFAmrmFwkXHGFsaWNlLWxh
YkBleGFtcGxlLnRlc3QACgkQAbyNRLiJR6BqAwv+J14wTa8mh+u/Va+TPaQjMJcK
RJL5aHtMWmC2IJy5wC52QMkQHCC5P8pkzJhrha3TEL/56wFiJdlDSn/dMN9i+m1I
Um08JNJtbMv2v7vAwWEHmdDHpYPgZZwwWz/C0Es3Ry6V+eBykSr+pjmMgfyMQsQ4
RqzE4MctDY7K6fB7oZiKr6NwBoSnxKgK+MNJXKx3Y2vy9sBm6tiVD6/cuJ4cxRwu
j6jOa5zSWXF3zMMOAg745Qb8XTPfpw5mO08i99kdjVie4xPOKo+Xt7vjBPO8gArF
l03OR4+oRPvskDg+5W/jY9GF9KrcVwhOi+Z7CH3+8+XLESEXUE4DZH0nwlbnwUxh
/pkq79efxwIkwFZ0fJMqqUApHVKPTBc4R4dbne63JCIV2YIMOS5V3BnTx98j88qz
FOF7lGBv7xVUdVe7eTZeljgQ0vb4eNAWD9/uOo8lMt7dTom+DOiQM4JLOUvGXuKk
++0F/gvOcOiGomQjPd5e8mj9YYq8iLxEj6ca8MtJ
=oiBr
-----END PGP SIGNATURE-----
root@vm:~/cns-pgp-smime$
```

Figure 10: Step B6: Bob verifies Alice's detached signature - Good signature (trust warning noted separately)

```
root@vm:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
root@vm:~/cns-pgp-smime$ gpg --verify ~/cns-pgp-smime/transfer/message.txt.asc ~/cns-pgp-smime/transfer/message.txt
gpg: Signature made Sat Oct 31 16:27:53 2026 UTC
gpg:                using RSA key B350D8CB3960E304F4ECD9CB01BC8D44B88947A0
gpg:                issuer "alice-lab@example.test"
gpg: Good signature from "Alice CNS Lab <alice-lab@example.test>" [unknown]
gpg: WARNING: The key's User ID is not certified with a trusted signature!
gpg:          There is no indication that the signature belongs to the owner.
Primary key fingerprint: B350 D8CB 3960 E304 F4EC  D9CB 01BC 8D44 B889 47A0
root@vm:~/cns-pgp-smime$ |
```

Figure 11: Step B7: Alice encrypts for Bob; Bob decrypts with his private key

```
root@vm:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg
root@vm:~/cns-pgp-smime$ gpg --armor --encrypt --recipient bob-lab@example.test --output ~/cns-pgp-smime/transfer/me
ssage.txt.pgp ~/cns-pgp-smime/message.txt
gpg: A4EEA39E12CE23D3: There is no assurance this key belongs to the named user

sub  rsa3072/A4EEA39E12CE23D3 2026-10-31 Bob CNS Lab <bob-lab@example.test>
 Primary key fingerprint: 7A85 0240 F9EC A7B0 28E8  2D4C C190 5678 CBE1 BC88
      Subkey fingerprint: 5DEF 144C 61CE E25E F0DE  4AD7 A4EE A39E 12CE 23D3

It is NOT certain that the key belongs to the person named
in the user ID.  If you *really* know what you are doing,
you may answer the next question with yes.

Use this key anyway? (y/N) y
root@vm:~/cns-pgp-smime$ head -n 6 ~/cns-pgp-smime/transfer/message.txt.pgp
-----BEGIN PGP MESSAGE-----

hQGMA6Tuo54SziPTAQv+Mcq3MnXdSwRx59ceyk9PFXk6dlXcX92fLxpVAWkahf4L
KLogTckq9wR0OxWC+83Cs2EdRtBeJ+xXD959G1H4Nm0TrQpRxCm2XV4O4grCr9N9
xMsazWWN1XxyguP2JCl6hl6LccgCnWxxYHawHTid2bPeWnjBEg1q0fu1g6djfZ1v
qV0veYdM/T8Mr5PFLibkL23joKNc1v7xlehTqA0EKQsFfW8LtL2ggvKHw9RgVNi9
root@vm:~/cns-pgp-smime$ export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg
root@vm:~/cns-pgp-smime$ gpg --output ~/cns-pgp-smime/decrypted_message.txt --decrypt ~/cns-pgp-smime/transfer/messa
ge.txt.pgp
gpg: encrypted with rsa3072 key, ID A4EEA39E12CE23D3, created 2026-10-31
      "Bob CNS Lab <bob-lab@example.test>"
Enter passphrase:
root@vm:~/cns-pgp-smime$ cat ~/cns-pgp-smime/decrypted_message.txt
CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01.
root@vm:~/cns-pgp-smime$ |
```

Figure 12: Step B7: signed-and-encrypted message; Bob decrypts and sees the signature status

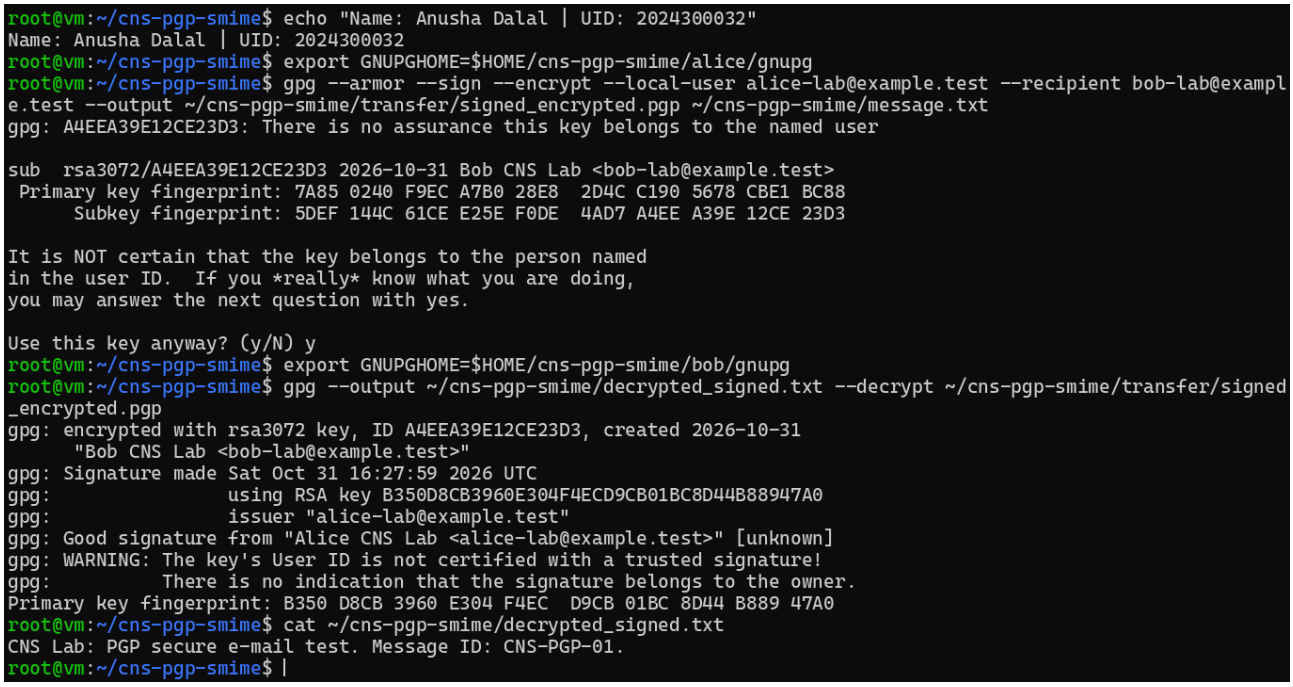


Figure 13: Step B8: tampered message checked against the original signature - BAD signature

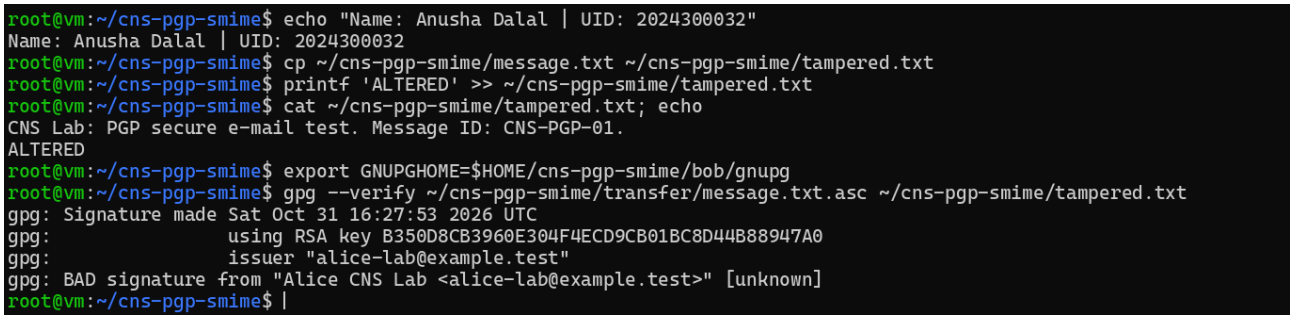
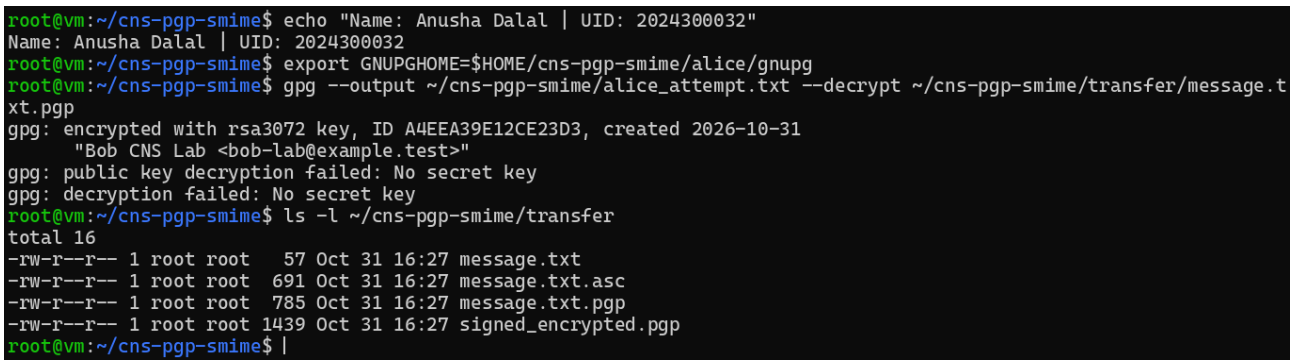


Figure 14: Table 2 (optional row): decryption with a non-matching private key fails; transfer folder contents



4. S/MIME evidence (Part C)

A local lab CA issues X.509 certificates for Alice and Bob; OpenSSL's smime command signs, verifies, encrypts and decrypts the message.

Figure 15: Step C1: OpenSSL version and S/MIME workspace

```
root@vm:~/cns-pgp-smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
root@vm:~/cns-pgp-smime$ mkdir -p ~/cns-pgp-smime/smime
root@vm:~/cns-pgp-smime$ cd ~/cns-pgp-smime/smime
root@vm:~/cns-pgp-smime$ |
```

Figure 16: Step C2: lab CA certificate (CA:TRUE, keyCertSign, cRLSign)

```
root@vm:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime/smime$ openssl req -x509 -newkey rsa:3072 -keyout ca.key -out ca.crt -sha256 -days 365 -node
s -subj '/CN=CNS Lab CA' -addext 'basicConstraints=critical,CA:TRUE' -addext 'keyUsage=critical,keyCertSign,cRLSign'
.....+....+......+...+..+...+.+...+..+...++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.....+...
......+...+..++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.........+...+...........+.........
............+..........+.....+.+......+.........+.........+...+........+.+........+.+..+...+.+.....................
...........+...........+...........+...+...........+....+.....+.................+.+......+...++++++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++
.............+..........+.........+.+..+...+...+.+..................+...+..+.+..+............+.+.....+....+........
...+.......+...+............++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.+.+.....++++++++++++++
++++++++++++++++++++++++++++++++++++++++++++++++++*.............................+..++++++++++++++++++++++++++++++
++++++++++++++++++++++++++++++++++
-----
root@vm:~/cns-pgp-smime/smime$ openssl x509 -in ca.crt -noout -subject -issuer -dates -ext basicConstraints,keyUsage
subject=CN = CNS Lab CA
issuer=CN = CNS Lab CA
notBefore=Oct 31 16:28:00 2026 GMT
notAfter=Oct 31 16:28:00 2027 GMT
X509v3 Basic Constraints: critical
    CA:TRUE
X509v3 Key Usage: critical
    Certificate Sign, CRL Sign
root@vm:~/cns-pgp-smime/smime$ |
```

Figure 17: Step C3: Alice's key, CSR and certificate signed by the lab CA

```
root@vm:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime/smime$ openssl req -newkey rsa:2048 -keyout alice.key -out alice.csr -nodes -subj '/CN=Alice
 CNS Lab/emailAddress=alice-lab@example.test' -addext 'keyUsage=digitalSignature,keyEncipherment' -addext 'extendedK
eyUsage=emailProtection'
...++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.......+...+.+......+...+...........+....+..+..
.......+...++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*..+........+....+.....................
.+.......+.....+.++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
..............+..+.........+.+...+.....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*.+......+.
.......+.+..+...+.+.....+.+..+............+.++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++*....+.
.....++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
-----
root@vm:~/cns-pgp-smime/smime$ openssl x509 -req -in alice.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out alice.c
rt -days 180 -sha256
Certificate request self-signature ok
subject=CN = Alice CNS Lab, emailAddress = alice-lab@example.test
root@vm:~/cns-pgp-smime/smime$ |
```

Figure 18: Step C4: Bob's key, CSR and certificate; subject, issuer and validity of both certificates

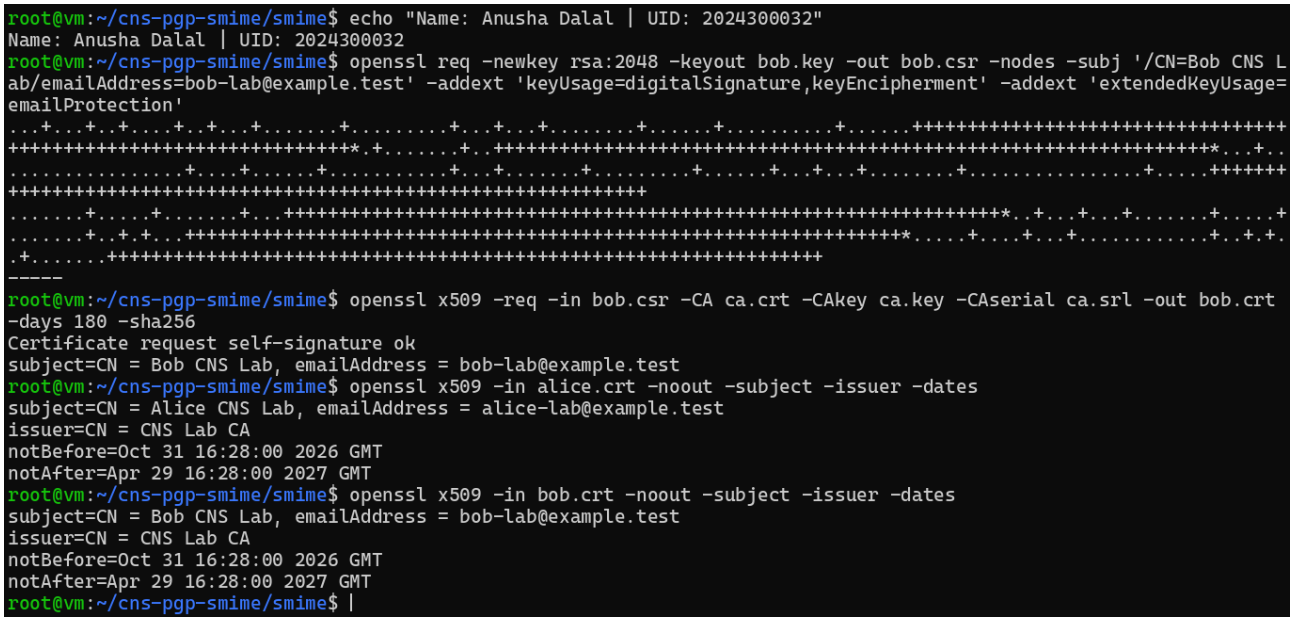


Figure 19: Certificate details: openssl x509 -text of Alice's certificate

```
root@vm:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime/smime$ openssl x509 -in alice.crt -noout -text
Certificate:
    Data:
        Version: 1 (0x0)
        Serial Number:
            7f:f7:5b:8d:46:c7:ab:6b:a6:3d:df:51:60:0f:c0:e2:43:89:b1:de
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: CN = CNS Lab CA
        Validity
            Not Before: Oct 31 16:28:00 2026 GMT
            Not After : Apr 29 16:28:00 2027 GMT
        Subject: CN = Alice CNS Lab, emailAddress = alice-lab@example.test
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:ae:cd:7e:7e:2b:f4:47:46:66:f9:f2:75:ac:d7:
                    cf:a4:19:da:dc:ac:8e:42:0e:40:47:1e:39:14:79:
                    e8:57:b0:25:20:20:0d:a8:cd:2b:cf:72:16:9c:95:
                    33:54:cb:31:59:13:8e:b1:93:8f:9c:c0:db:35:f8:
                    15:ea:72:a6:08:32:b6:4d:99:f3:4a:84:c8:af:c5:
                    51:34:2b:38:a7:55:80:b4:d0:7b:cd:cc:18:a3:82:
                    36:d1:52:bf:36:ce:31:10:f3:b5:e3:3a:da:33:aa:
                    c7:fa:46:35:c3:8d:29:e9:18:7a:8c:d4:a6:ce:df:
                    c2:75:57:01:7f:9c:c7:80:8e:4c:fd:a4:11:3a:ba:
                    fd:d7:81:24:2c:ea:30:77:f4:7a:ab:d5:54:ef:cc:
                    8a:98:06:53:b3:74:c7:f0:c9:07:79:c7:e1:77:56:
                    f6:73:ce:60:cb:0e:b6:22:5f:af:10:0a:38:5c:55:
                    a6:93:37:47:32:c2:8c:01:56:13:e9:92:85:a3:15:
                    fb:ab:35:e7:60:5f:8d:a5:cc:35:22:ab:a8:70:ad:
                    74:4f:97:17:75:7a:8f:21:fc:84:b5:10:a5:9f:86:
                    68:52:cb:54:96:36:b3:31:0b:88:d0:36:34:9f:19:
                    46:aa:5d:15:61:50:5b:b9:0d:3c:bd:4a:82:0e:92:
                    7c:47
                Exponent: 65537 (0x10001)
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        6a:c4:7b:98:41:eb:46:24:c1:88:69:17:f7:3f:23:37:7c:27:
        c0:c4:89:c1:fe:3f:ce:aa:df:f7:a0:b9:f6:16:13:3a:c7:bb:
        5f:dd:b0:9f:c8:bb:bd:5c:ce:8e:f7:4d:1d:3b:72:f8:4f:0d:
        98:1b:56:e8:04:63:8c:bc:a2:53:87:ee:08:a9:17:06:11:d0:
        c2:fb:0a:2e:b6:c5:a0:8f:e4:62:18:ce:36:87:42:8e:35:8e:
        32:2f:bb:2c:85:2e:86:c7:86:2b:2d:24:21:17:62:9b:6d:38:
        be:92:67:71:3e:5e:07:f3:fd:0e:fa:1d:eb:c5:d1:a2:07:59:
        5d:59:11:99:ac:b6:97:3d:c6:d9:d1:86:ed:92:98:c7:30:98:
        0f:b7:98:2d:d8:18:43:dd:11:f4:b0:74:9f:ed:48:41:86:36:
        20:2b:51:49:6d:13:42:ae:00:1e:73:d2:d6:c5:19:b8:a8:bb:
        51:44:68:8c:7e:15:75:2d:06:20:6d:b0:34:03:b9:9a:ce:49:
        c5:99:c7:27:35:5e:87:8d:04:42:e1:31:08:fc:ff:1f:6d:5b:
        7c:21:ee:71:8a:6e:01:cc:46:11:dd:1e:0e:b2:91:fe:0b:fe:
        76:a5:9b:85:cb:42:0f:63:23:95:d0:f7:f9:0a:21:3e:da:12:
        dd:c0:03:78:8d:a1:2e:6f:31:18:05:a1:66:1f:a2:04:28:a1:
        ca:67:77:f8:f6:5f:34:05:4b:ee:59:fd:34:c0:f2:59:8a:9d:
        77:2b:9c:15:0a:14:b8:ac:68:7c:67:e4:d8:85:78:a0:d5:f6:
        33:3a:4e:66:da:25:52:d5:2c:b8:f8:75:13:c8:55:85:12:c9:
        1b:28:e0:7a:e0:32:fa:70:f4:a5:a7:34:dd:98:f8:43:b7:ff:
        4f:a8:e1:e8:40:a9:61:11:29:a6:e6:58:1b:86:0f:cc:42:12:
        b7:e9:89:7e:3c:91:05:cb:b8:04:b0:ed:3d:f3:f8:3e:f1:1d:
        78:2d:9e:0d:0c:03
root@vm:~/cns-pgp-smime/smime$
```

Figure 20: Step C5: S/MIME signed message (first 15 lines)

```
root@vm:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime/smime$ printf 'CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.\n' > message.tx
t
root@vm:~/cns-pgp-smime/smime$ openssl smime -sign -in message.txt -text -signer alice.crt -inkey alice.key -out sig
ned.eml
root@vm:~/cns-pgp-smime/smime$ head -n 15 signed.eml
MIME-Version: 1.0
Content-Type: multipart/signed; protocol="application/x-pkcs7-signature"; micalg="sha-256"; boundary="----DB346FE825
1561208FCA8F72AD20EFF7"

This is an S/MIME signed message

------DB346FE8251561208FCA8F72AD20EFF7
Content-Type: text/plain

CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.

------DB346FE8251561208FCA8F72AD20EFF7
Content-Type: application/x-pkcs7-signature; name="smime.p7s"
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="smime.p7s"
root@vm:~/cns-pgp-smime/smime$
```

Figure 21: Step C6: signed message verified with the lab CA - Verification successful

```
root@vm:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime/smime$ openssl smime -verify -in signed.eml -CAfile ca.crt -out verified_message.txt
Verification successful
root@vm:~/cns-pgp-smime/smime$ cat verified_message.txt
Content-Type: text/plain

CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
root@vm:~/cns-pgp-smime/smime$
```

Figure 22: Steps C7-C8: encrypted for Bob (enveloped data) and decrypted with bob.key

```
root@vm:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime/smime$ openssl smime -encrypt -in message.txt -text -out encrypted.eml bob.crt
root@vm:~/cns-pgp-smime/smime$ head -n 12 encrypted.eml
MIME-Version: 1.0
Content-Disposition: attachment; filename="smime.p7m"
Content-Type: application/x-pkcs7-mime; smime-type=enveloped-data; name="smime.p7m"
Content-Transfer-Encoding: base64

MIIB6QYJKoZIhvcNAQcDoIIB2jCCAdYCAQAxggFJMIIBRQIBADAtMBUxEzARBgNV
BAMMCkNOUyBMYWIgQ0ECFH/3W41Gx6trpj3fUWAPwOJDibHfMA0GCSqGSIb3DQEB
AQUABIIBAHCl9EffmLjtT6yOLGrVl3UrOj4smbTyFMM/14hq+La8IjPZqPReO/BW
RktY50xQxOXAfOV91RAdxF4Brq+iCQOv3+2Lii0bH6oR6HDAdK/7fZZaG/kJYydS
y8cEz+FSTSCLuGbhbMAzkRYn2eCcx1SIahlAFa7t6BNxD37Ww4IX4TF7tk2AYOa/
tf3V4dtmvk/b9cqWFt/W7JC0qQuib8rBgk4IBG405/9NRcZrkLcLtvuS9Gbp5pfb
y1SIkh+hPq5qbPSrc8aRlFBaU/vWI6xy1XJ9d57U4uwxUvNjkZ+irZ6r7mt7TTC0
root@vm:~/cns-pgp-smime/smime$ openssl smime -decrypt -in encrypted.eml -recip bob.crt -inkey bob.key -out smime_dec
rypted.txt
root@vm:~/cns-pgp-smime/smime$ cat smime_decrypted.txt
Content-Type: text/plain

CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
root@vm:~/cns-pgp-smime/smime$ diff -u message.txt smime_decrypted.txt
--- message.txt 2026-10-31 16:28:00.627548845 +0000
+++ smime_decrypted.txt 2026-10-31 16:28:00.686483106 +0000
@@ -1 +1,3 @@
-CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
+Content-Type: text/plain
+
+CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
root@vm:~/cns-pgp-smime/smime$ tail -n +3 smime_decrypted.txt | tr -d '\r' | diff - message.txt && echo 'message bod
y is identical to message.txt'
message body is identical to message.txt
root@vm:~/cns-pgp-smime/smime$
```

Figure 23: Step C9: sign then encrypt; Bob decrypts and verifies the recovered signed message

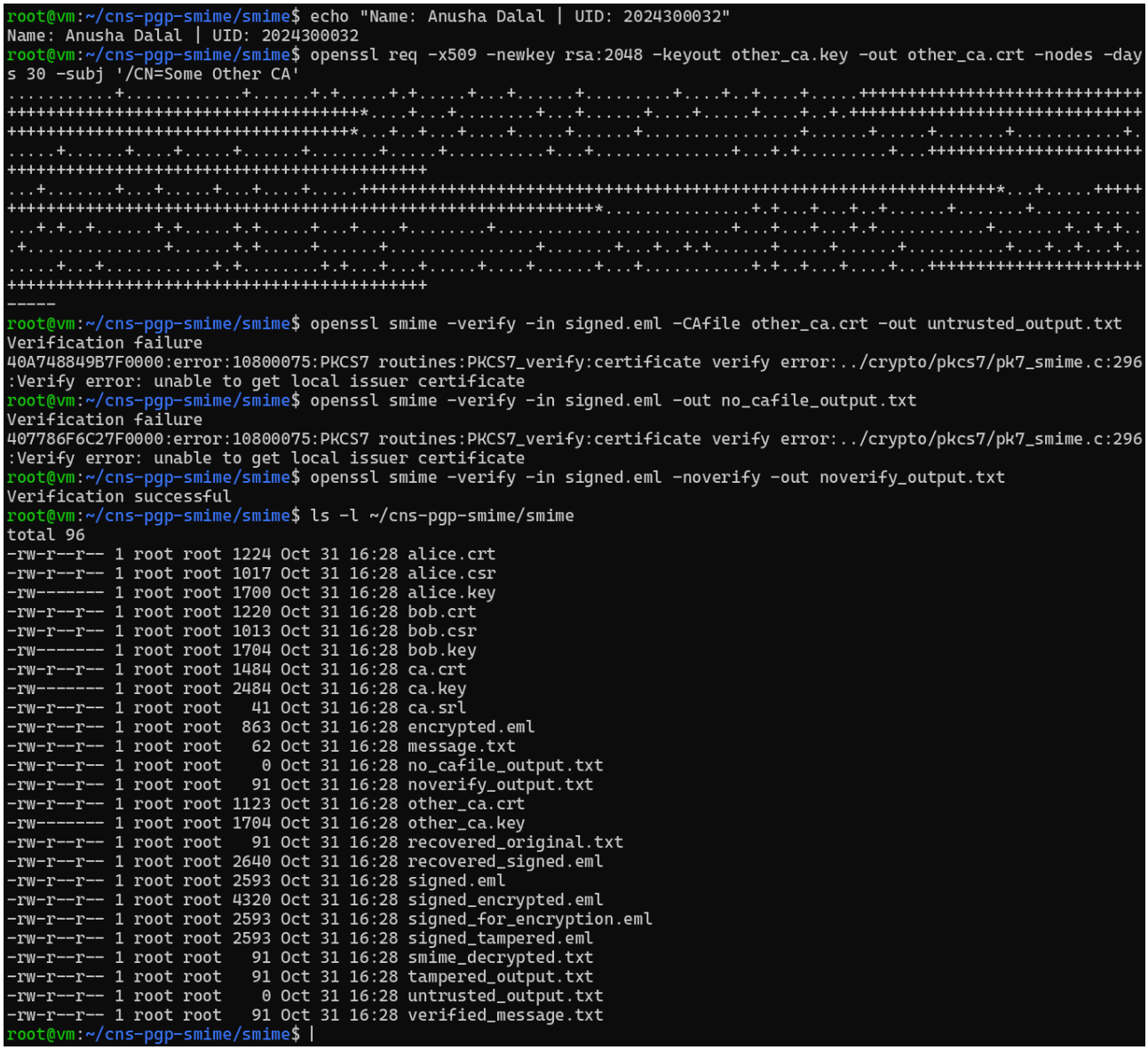
```
root@vm:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime/smime$ openssl smime -sign -in message.txt -text -signer alice.crt -inkey alice.key -out sig
ned_for_encryption.eml
root@vm:~/cns-pgp-smime/smime$ openssl smime -encrypt -in signed_for_encryption.eml -out signed_encrypted.eml bob.cr
t
root@vm:~/cns-pgp-smime/smime$ head -n 8 signed_encrypted.eml
MIME-Version: 1.0
Content-Disposition: attachment; filename="smime.p7m"
Content-Type: application/x-pkcs7-mime; smime-type=enveloped-data; name="smime.p7m"
Content-Transfer-Encoding: base64

MIIL5AYJKoZIhvcNAQcDoIIL1TCCC9ECAQAxggFJMIIBRQIBADAtMBUxEzARBgNV
BAMMCkNOUyBMYWIgQ0ECFH/3W41Gx6trpj3fUWAPwOJDibHfMA0GCSqGSIb3DQEB
AQUABIIBAIgbPMS/PZjwIFDwE+i1VqhynfdhzocYCKCsgZEOg4KbD5Vu03J2sCeg
root@vm:~/cns-pgp-smime/smime$ openssl smime -decrypt -in signed_encrypted.eml -recip bob.crt -inkey bob.key -out re
covered_signed.eml
root@vm:~/cns-pgp-smime/smime$ openssl smime -verify -in recovered_signed.eml -CAfile ca.crt -out recovered_original
.txt
Verification successful
root@vm:~/cns-pgp-smime/smime$ tail -n +3 recovered_original.txt | tr -d '\r' | diff - message.txt && echo 'message
body is identical to message.txt'
message body is identical to message.txt
root@vm:~/cns-pgp-smime/smime$
```

Figure 24: Step C10: one character of the signed body changed - Verification failure (digest failure)

```
root@vm:~/cns-pgp-smime/smime$ echo "Name: Anusha Dalal | UID: 2024300032"
Name: Anusha Dalal | UID: 2024300032
root@vm:~/cns-pgp-smime/smime$ cp signed.eml signed_tampered.eml
root@vm:~/cns-pgp-smime/smime$ sed -i 's/S\/MIME secure e-mail test/S\/MIME secure e-mail tost/' signed_tampered.eml
root@vm:~/cns-pgp-smime/smime$ diff signed.eml signed_tampered.eml
9c9
< CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.
---
> CNS Lab: S/MIME secure e-mail tost. Message ID: CNS-SMIME-01.
root@vm:~/cns-pgp-smime/smime$ openssl smime -verify -in signed_tampered.eml -CAfile ca.crt -out tampered_output.txt
Verification failure
4067FD7E477F0000:error:10800065:PKCS7 routines:PKCS7_signatureVerify:digest failure:../crypto/pkcs7/pk7_doit.c:1096:
4067FD7E477F0000:error:10800069:PKCS7 routines:PKCS7_verify:signature failure:../crypto/pkcs7/pk7_smime.c:345:
root@vm:~/cns-pgp-smime/smime$
```

Figure 25: Step C10: wrong CA / no CA file - certificate trust failure; with -noverify the signature itself is valid



5. Results, comparative analysis, discussion and conclusion

Table 2: PGP Results

Test	Expected result	Observed result / evidence
Public key export/import	Keys imported; fingerprints can be inspected	Both keys exported as ASCII-armored .asc files and imported into the other keyring ("public key imported"); fingerprints shown with `gpg --fingerprint`: Alice B350 D8CB 3960 E304 F4EC D9CB 01BC 8D44 B889 47A0, Bob 7A85 0240 F9EC A7B0 28E8 2D4C C190 5678 CBE1 BC88
Detached signature on original message	Good signature (trust status noted separately)	**Good signature from "Alice CNS Lab <alice-lab@example.test>" [unknown]**; separately: WARNING, the key is not certified with a trusted signature
Encrypt for Bob and decrypt as Bob	Decryption succeeds; content matches original	Decrypted with Bob's rsa3072 subkey; `decrypted_message.txt` = "CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01."
Signed-and-encrypted message	Bob decrypts and signature status is displayed	Bob decrypted it and GnuPG showed **Good signature** from Alice (with the same trust warning); content matches
Modified message checked against original signature	Signature verification fails	**BAD signature from "Alice CNS Lab"** (tampered.txt had "ALTERED" appended)
Attempt to decrypt with a non-matching private key (optional)	Decryption fails	Alice tried to decrypt the message for Bob: **"decryption failed: No secret key"**

Table 3: S/MIME Results

Test	Expected result	Observed result / evidence
Inspect Alice/Bob certificate	Subject, issuer and validity displayed	Subject CN=Alice CNS Lab / CN=Bob CNS Lab with their e-mail addresses; issuer CN=CNS Lab CA; valid 180 days (Oct 31 16:28:00 2026 GMT to Apr 29 16:28:00 2027 GMT)
Verify original signed message using lab CA	Verification succeeds	**Verification successful**; recovered text written to verified_message.txt
Encrypt for Bob and decrypt using bob.key	Decryption succeeds; content matches original	encrypted.eml is enveloped-data (smime.p7m); decryption succeeded; message body identical to message.txt (only the MIME header added by `-text` and CRLF line endings differ)
Sign and encrypt workflow	Bob recovers the signed message	Bob decrypted signed_encrypted.eml, **Verification successful** on the recovered signed message, body identical to message.txt
Modified signed content	Signature verification fails	**Verification failure**: `PKCS7_signatureVerify: digest failure` and `signature failure` (one character changed: "test" → "tost")
Verify with untrusted/missing CA	Certificate trust validation fails or warns	With another CA or no -CAfile: **Verification failure: unable to get local issuer certificate** (a trust failure). With `-noverify` the same message verifies, so the signature itself is valid

Comparative analysis

Parameter	PGP / OpenPGP	S/MIME
Key / identity representation	OpenPGP public key and fingerprint	X.509 certificate containing public key and identity fields
Trust model	Often decentralized; fingerprints and user certifications	Usually CA-issued certificate chain and validation
Typical message representation	OpenPGP packet / ASCII-armored data	MIME with CMS/PKCS #7 structures
Confidentiality mechanism	Hybrid encryption; session key protected for recipient	Hybrid encryption; content key protected for recipient
Signature verification requires	Correct sender public key and valid signature	Sender certificate/public key, valid signature and applicable trust checks
Common deployment	OpenPGP-compatible clients and workflows	Enterprise e-mail and certificate-based environments
Observed challenge	Public-key authenticity and trust decisions	Certificate issuance, chain validation and lifecycle management

PGP was easier to set up in the lab: each user generated a key pair and exchanged public keys directly, with no third party. Its extra trust step is manual: each user must compare the full key fingerprint over a separate trusted channel (and could certify the key), otherwise GnuPG warns that the key is not certified. S/MIME needed more setup, because a CA had to be created first and had to issue a certificate for each user. In return, the recipient only has to trust the CA certificate, after which every certificate it issued can be validated automatically, but the CA's issuance, chain validation, renewal and revocation must be managed.

Answers to the discussion questions

1. PGP vs S/MIME: key management and trust model.

PGP users create and exchange their own key pairs, and trust is decentralised: you decide who to trust by checking fingerprints or through signatures from people you trust (web of trust). S/MIME uses X.509 certificates issued by a Certificate Authority; trust is hierarchical, so you trust the CA and therefore every certificate in its chain.

2. Why hybrid encryption?

Symmetric encryption (e.g. AES) is fast and suits large message content, but needs a shared key. Public-key encryption solves key distribution but is slow and limited in size. So a random one-time session key encrypts the message, and only that small key is encrypted with the recipient's public key.

3. Signing and verifying keys.

The sender signs with **her own private key**; anyone verifies with the **sender's public key** (in S/MIME, the public key inside her certificate).

4. Encrypting for Bob and decrypting.

The message is encrypted with **Bob's public key** (his certificate in S/MIME); Bob decrypts with **his own private key**.

5. Why never send private keys?

Whoever has a private key can decrypt everything sent to that person and sign messages in their name. The transfer folder and e-mail are insecure channels, so a sent private key must be assumed compromised; confidentiality, authentication and non-repudiation would all be lost.

6. Valid signature vs trusted identity.

A valid signature only proves the message was signed by the private key matching the public key used, and was not changed. It does not prove whose key it is. Trust in the identity needs separate checking: fingerprint verification

(PGP) or a valid CA chain (S/MIME). That is why GnuPG said "Good signature" and also "not certified with a trusted signature".

7. Why import without fingerprint check enables MITM.

An attacker on the channel could replace Alice's public key with their own key carrying Alice's name. Bob would then accept the attacker's signatures as Alice's, and messages Bob encrypts "to Alice" could be read by the attacker. Comparing the full fingerprint over a trusted channel detects the substitution.

8. Role of the CA certificate in S/MIME verification.

It is the trust anchor. OpenSSL checks that Alice's certificate was signed by a CA in the -CAfile; with the CA it reported "Verification successful", and without it "unable to get local issuer certificate", even though the signature itself was valid.

9. What happened when the message was modified?

Verification failed: GnuPG reported "BAD signature" and OpenSSL "digest failure". The signature is over a hash of the exact content; any change gives a different hash that no longer matches the signed hash, so integrity violation is detected.

10. Why is a lab/self-signed certificate not trusted in production?

Mail clients trust only CAs in their built-in trust store, which are audited and verify identities. Our lab CA is not in that store, and anyone can create a self-signed CA claiming any name, so trusting it automatically would let attackers forge identities.

11. Can a signed message also be encrypted?

Yes. Sign first, then encrypt: the signature covers the real content and is itself hidden inside the encryption, so only Bob sees who signed and what was signed. Bob decrypts first, then verifies. We did this in both parts (`--sign --encrypt` in PGP; sign then encrypt in S/MIME).

12. One advantage and one limitation of each.

- **PGP:** advantage: free, no CA needed, users control their own keys. Limitation: key authenticity depends on users checking fingerprints; weak usability for non-experts.
- **S/MIME:** advantage: built into enterprise mail clients, and automatic validation through a CA chain. Limitation: requires obtaining, renewing and revoking certificates from a CA (cost and management overhead).

Conclusion (150–200 words)

Conclusion
[hand-written page attached]

6. Commands used and deviations

Commands used

Step	Commands

Table 1	`lsb_release -d 2>/dev/null || grep PRETTY_NAME /etc/os-release`
	`uname -r`
	`gpg --version | head -3`
	`openssl version`
	`lscpu | grep -E 'Model name|^CPU\(s\)'`
	`free -h | head -2`
	`date`

B1	`gpg --version`

B2	`mkdir -p ~/cns-pgp-smime/{alice,bob,transfer}`
	`cd ~/cns-pgp-smime`
	`mkdir -p ~/cns-pgp-smime/alice/gnupg ~/cns-pgp-smime/bob/gnupg`
	`chmod 700 ~/cns-pgp-smime/alice/gnupg ~/cns-pgp-smime/bob/gnupg`
	`echo "pinentry-mode loopback" > ~/cns-pgp-smime/alice/gnupg/gpg.conf`
	`echo "pinentry-mode loopback" > ~/cns-pgp-smime/bob/gnupg/gpg.conf`
	`ls -la ~/cns-pgp-smime ~/cns-pgp-smime/alice ~/cns-pgp-smime/bob`

B3 (Alice)	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --full-generate-key`
	`gpg --list-keys`
	`gpg --list-secret-keys`

B3 (Bob)	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --full-generate-key`
	`gpg --list-keys`

B4	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --armor --export alice-lab@example.test > ~/cns-pgp-smime/alice_public.asc`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --armor --export bob-lab@example.test > ~/cns-pgp-smime/bob_public.asc`
	`ls -l ~/cns-pgp-smime/*.asc`
	`head -n 4 ~/cns-pgp-smime/alice_public.asc`
	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --import ~/cns-pgp-smime/bob_public.asc`
	`gpg --fingerprint bob-lab@example.test`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --import ~/cns-pgp-smime/alice_public.asc`
	`gpg --fingerprint alice-lab@example.test`

B5	`printf 'CNS Lab: PGP secure e-mail test. Message ID: CNS-PGP-01.\n' >`
	`~/cns-pgp-smime/message.txt`
	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --armor --local-user alice-lab@example.test --detach-sign --output`
	`~/cns-pgp-smime/message.txt.asc ~/cns-pgp-smime/message.txt`
	`cp ~/cns-pgp-smime/message.txt* ~/cns-pgp-smime/transfer/`
	`cat ~/cns-pgp-smime/transfer/message.txt ~/cns-pgp-smime/transfer/message.txt.asc`

B6	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --verify ~/cns-pgp-smime/transfer/message.txt.asc`
	`~/cns-pgp-smime/transfer/message.txt`

B7	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --armor --encrypt --recipient bob-lab@example.test --output`
	`~/cns-pgp-smime/transfer/message.txt.pgp ~/cns-pgp-smime/message.txt`
	`head -n 6 ~/cns-pgp-smime/transfer/message.txt.pgp`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --output ~/cns-pgp-smime/decrypted_message.txt --decrypt`
	`~/cns-pgp-smime/transfer/message.txt.pgp`
	`cat ~/cns-pgp-smime/decrypted_message.txt`

B7 (sign + encrypt)	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --armor --sign --encrypt --local-user alice-lab@example.test --recipient bob-lab@example.test --output ~/cns-pgp-smime/transfer/signed_encrypted.pgp ~/cns-pgp-smime/message.txt`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --output ~/cns-pgp-smime/decrypted_signed.txt --decrypt`
	`~/cns-pgp-smime/transfer/signed_encrypted.pgp`
	`cat ~/cns-pgp-smime/decrypted_signed.txt`

B8	`cp ~/cns-pgp-smime/message.txt ~/cns-pgp-smime/tampered.txt`
	`printf 'ALTERED' >> ~/cns-pgp-smime/tampered.txt`
	`cat ~/cns-pgp-smime/tampered.txt; echo`
	`export GNUPGHOME=$HOME/cns-pgp-smime/bob/gnupg`
	`gpg --verify ~/cns-pgp-smime/transfer/message.txt.asc ~/cns-pgp-smime/tampered.txt`

Table 2 (optional)	`export GNUPGHOME=$HOME/cns-pgp-smime/alice/gnupg`
	`gpg --output ~/cns-pgp-smime/alice_attempt.txt --decrypt`
	`~/cns-pgp-smime/transfer/message.txt.pgp`
	`ls -l ~/cns-pgp-smime/transfer`

C1	`openssl version`
	`mkdir -p ~/cns-pgp-smime/smime`
	`cd ~/cns-pgp-smime/smime`

C2	`openssl req -x509 -newkey rsa:3072 -keyout ca.key -out ca.crt -sha256 -days 365 -nodes -subj '/CN=CNS Lab CA' -addext 'basicConstraints=critical,CA:TRUE' -addext 'keyUsage=critical,keyCertSign,cRLSign'` `openssl x509 -in ca.crt -noout -subject -issuer -dates -ext basicConstraints,keyUsage`
C3	`openssl req -newkey rsa:2048 -keyout alice.key -out alice.csr -nodes -subj '/CN=Alice CNS Lab/emailAddress=alice-lab@example.test' -addext 'keyUsage=digitalSignature,keyEncipherment' -addext 'extendedKeyUsage=emailProtection'` `openssl x509 -req -in alice.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out alice.crt -days 180 -sha256`

C4	`openssl req -newkey rsa:2048 -keyout bob.key -out bob.csr -nodes -subj '/CN=Bob CNS Lab/emailAddress=bob-lab@example.test' -addext 'keyUsage=digitalSignature,keyEncipherment' -addext 'extendedKeyUsage=emailProtection'`
	`openssl x509 -req -in bob.csr -CA ca.crt -CAkey ca.key -CAserial ca.srl -out bob.crt -days 180 -sha256`
	`openssl x509 -in alice.crt -noout -subject -issuer -dates`
	`openssl x509 -in bob.crt -noout -subject -issuer -dates`

C4 (details)	`openssl x509 -in alice.crt -noout -text`

C5	`printf 'CNS Lab: S/MIME secure e-mail test. Message ID: CNS-SMIME-01.\n' > message.txt`
	`openssl smime -sign -in message.txt -text -signer alice.crt -inkey alice.key -out signed.eml`
	`head -n 15 signed.eml`

C6	`openssl smime -verify -in signed.eml -CAfile ca.crt -out verified_message.txt` `cat verified_message.txt`

C7–C8	`openssl smime -encrypt -in message.txt -text -out encrypted.eml bob.crt`
	`head -n 12 encrypted.eml`
	`openssl smime -decrypt -in encrypted.eml -recip bob.crt -inkey bob.key -out smime_decrypted.txt`
	`cat smime_decrypted.txt`
	`diff -u message.txt smime_decrypted.txt`
	`tail -n +3 smime_decrypted.txt | tr -d '\r' | diff - message.txt && echo 'message body is identical to message.txt'`

C9	`openssl smime -sign -in message.txt -text -signer alice.crt -inkey alice.key -out signed_for_encryption.eml`
	`openssl smime -encrypt -in signed_for_encryption.eml -out signed_encrypted.eml bob.crt`
	`head -n 8 signed_encrypted.eml`
	`openssl smime -decrypt -in signed_encrypted.eml -recip bob.crt -inkey bob.key -out recovered_signed.eml`
	`openssl smime -verify -in recovered_signed.eml -CAfile ca.crt -out recovered_original.txt`
	`tail -n +3 recovered_original.txt | tr -d '\r' | diff - message.txt && echo 'message body is identical to message.txt'`

C10 (tamper)	`cp signed.eml signed_tampered.eml`
	`sed -i 's/S\/MIME secure e-mail test/S\/MIME secure e-mail tost/' signed_tampered.eml`
	`diff signed.eml signed_tampered.eml`
	`openssl smime -verify -in signed_tampered.eml -CAfile ca.crt -out tampered_output.txt`

C10 (trust)	`openssl req -x509 -newkey rsa:2048 -keyout other_ca.key -out other_ca.crt -nodes -days 30 -subj '/CN=Some Other CA'`
	`openssl smime -verify -in signed.eml -CAfile other_ca.crt -out untrusted_output.txt`
	`openssl smime -verify -in signed.eml -out no_cafile_output.txt`
	`openssl smime -verify -in signed.eml -noverify -out noverify_output.txt`
	`ls -l ~/cns-pgp-smime/smime`

Deviations caused by software versions or the environment

- **Passphrase prompt in the terminal:** without a graphical desktop, GnuPG's pinentry pop-up is not available, so each keyring's gpg.conf contains `pinentry-mode loopback`; GnuPG then asks for the passphrase directly in the terminal (the passphrase is not echoed). The key-generation answers were exactly those in the manual (RSA and RSA, 3072 bits, expiry 1 year, the given names and e-mail addresses).
- **"Use this key anyway? (y/N)":** when Alice encrypts for Bob, GnuPG 2.4 asks this because Bob's imported key is not yet certified; it was answered y after the fingerprint was compared (Step B4).
- **C10 edit with sed instead of a text editor:** one character of the signed body was changed with `sed` ("test" to "tost"); `diff` shows that only that line changed and the MIME structure is intact.
- **Certificate extensions:** in OpenSSL 3.0, `openssl x509 -req` does not copy extensions from the CSR unless `-copy_extensions copy` is given, so alice.crt and bob.crt contain no keyUsage/extendedKeyUsage. S/MIME still works, because a certificate without these extensions is not restricted.

- **Decrypted file vs message.txt:** the `-text` option wraps the message in a `Content-Type: text/plain` MIME header and S/MIME uses CRLF line endings, so the files differ by the header and line endings only; the message body was compared with `tail -n +3 | tr -d '\r' | diff` and is identical.
- **Untrusted CA test:** in addition to omitting -CAfile, a second unrelated CA ("Some Other CA") was used, and `-noverify` was run to show that the failure is a certificate-trust failure, not a content failure.